Pertemuan 09
Binary Search Tree

Kuliah Struktur Data

Sekolah Vokasi IPB

Agenda

- Apa itu Binary Search Tree (BST) ?
- Operasi pada BST?
- Implementasi BST (tree.hh)
- Pemanfaatan BST

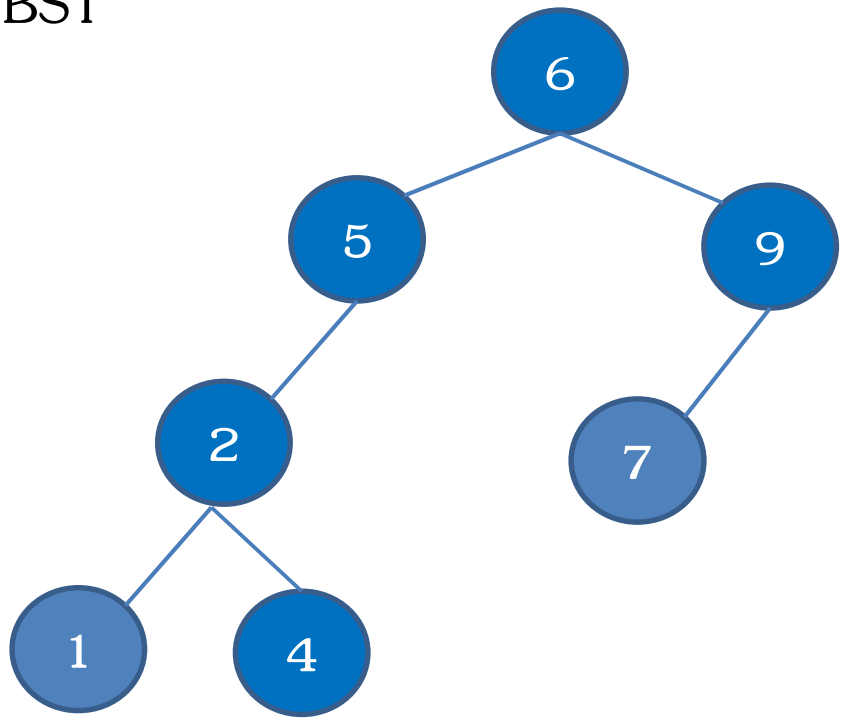
Apa itu Binary Search Tree?

Binary Search Tree (BST)

- Suatu struktur data tree yang pada setiap posisi nodenya memenuhi sifat **value anak kiri < value node < value anak kanan**
- Proses penelusuran suatu value tertentu pada tree secara sederhana dapat dilakukan mulai dari *root*:
  - jika ditemukan langsung ambil
  - bergerak ke kiri jika lebih kecil,
  - atau ke kanan jika lebih besar.
- Prosedur inipun berlaku untuk melakukan insert value baru pada *tree*

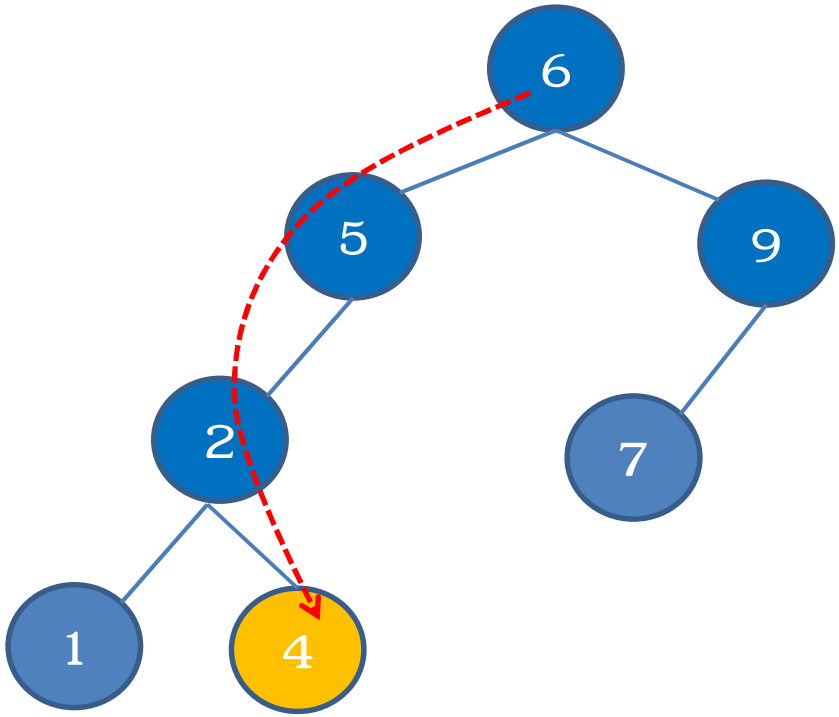
Contoh BST

- Contoh sebuah BST



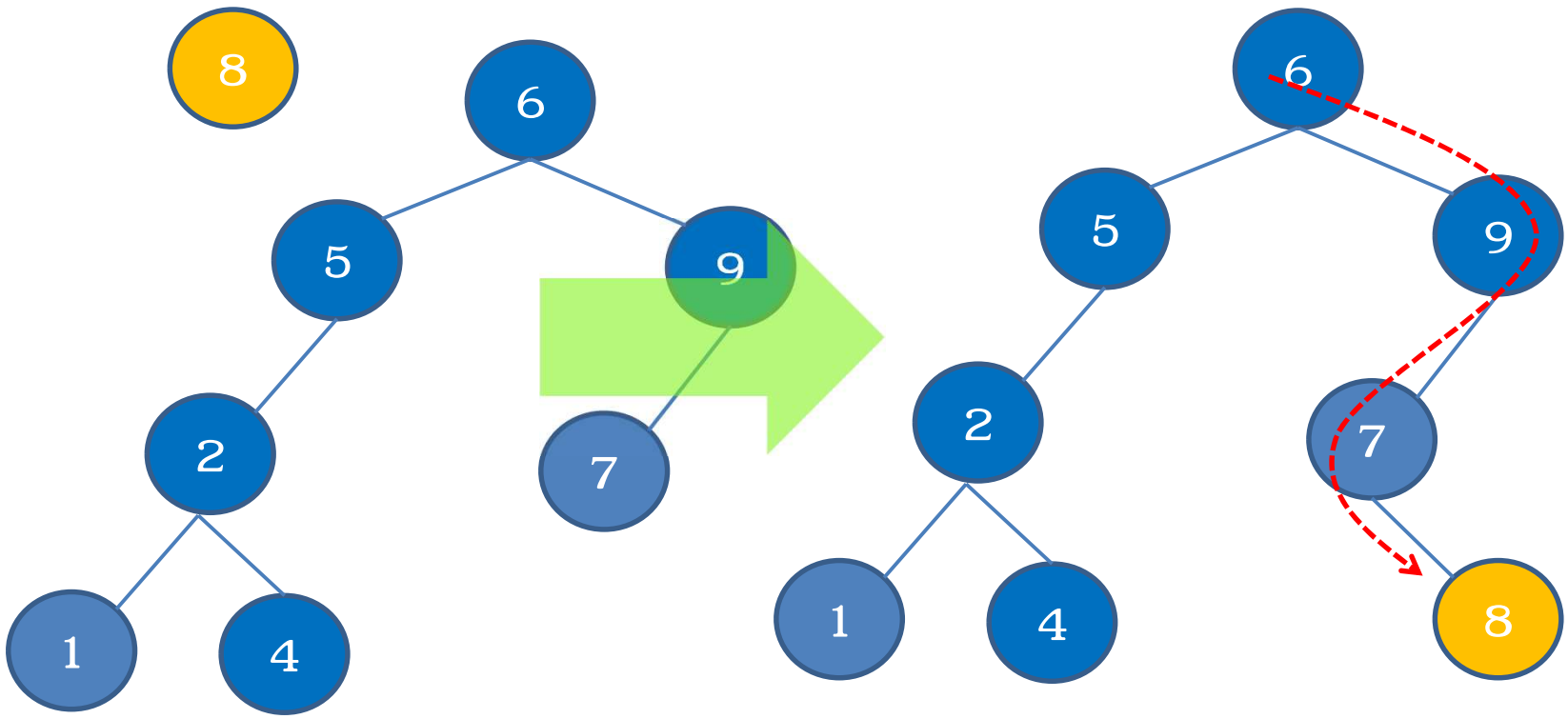
Ilustrasi penelusuran

- Mencari node dengan value **4**



Ilustrasi Insert node

- Memasukan node bernilai **8**



Operasi pada BST?

Operasi dasar

- Seperti halnya struktur data lain operasi dasar BST yang utama meliputi
  - Search, Insert. Update, dan Delete
- Search dan insert telah dijabarkan sebelumnya
- untuk update dapat diwujudkan sebagai penghapusan dahulu baru dilanjutkan insert node dengan value baru

Delete

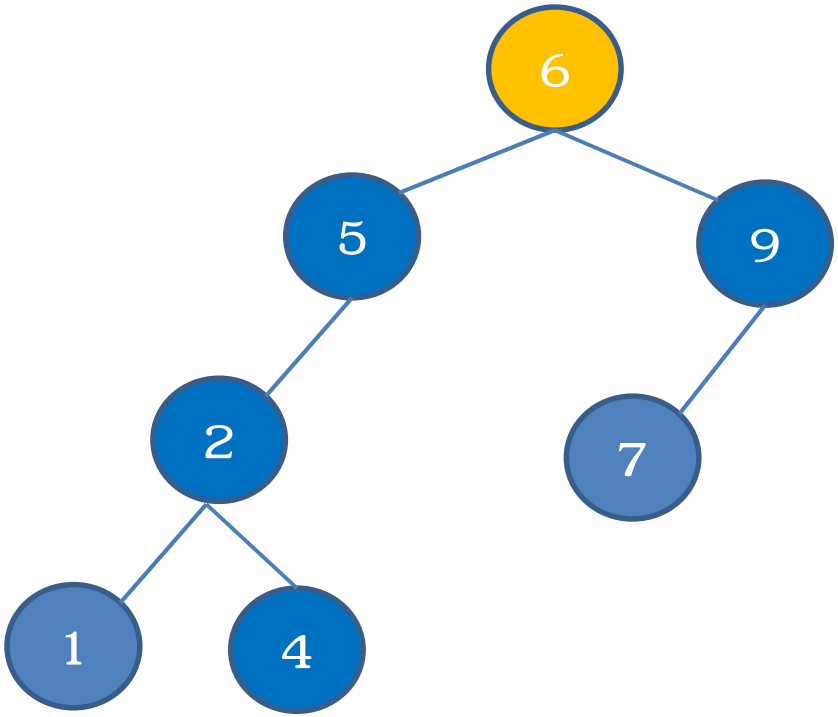
- Operasi delete akan dikategorisasikan kepada 3 kasus berdasar lokasi node yang dihapus:
  - Untuk node leaf langsung hapus
  - Jika node memiliki 1 anak maka jadikan anak yang satu-satunya tersebut sebagai pengganti node yang dihapus
  - Jika node memiliki 2 anak maka sebagai pengganti node yang dihapus pilih satu dari dua opsi (predesessor inorder atau sucessor inorder)

(Pred & Succ) inorder

- Predecessor inorder: node yang dikunjungi tepat sebelum node terkait dengan penelusuran inorder
- Successor inorder: node yang dikunjungi tepat setelah node terkait dengan penelusuran inorder
- Coba kenapa dipilih (Pred XOR Succ) inorder ini?
- **Agar seragam kesepakatan kita pilih predecessor inorder yah**

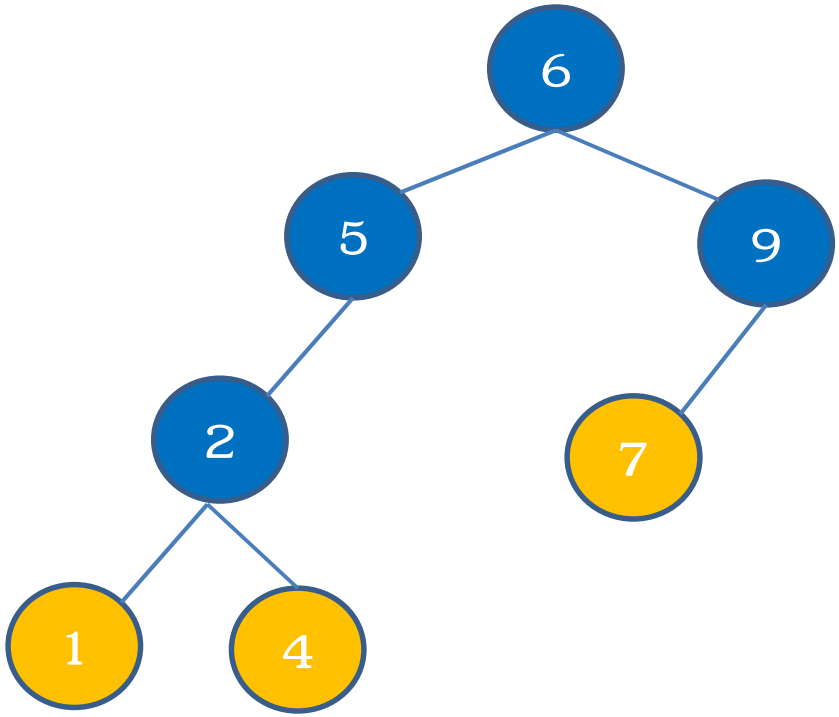
Ilustrasi Pred & In

- Inorder dari tree: 1 2 4 5 6 7 9
- Pred inorder 6 adalah 5
- Succ inorder 6 adalah 7



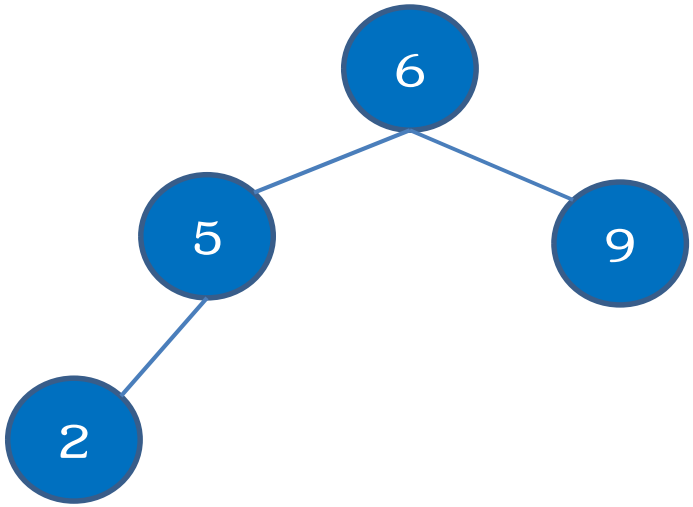
Ilustrasi hapus node leaf

- Menghapus node **1**, **4**, ataupun **7**



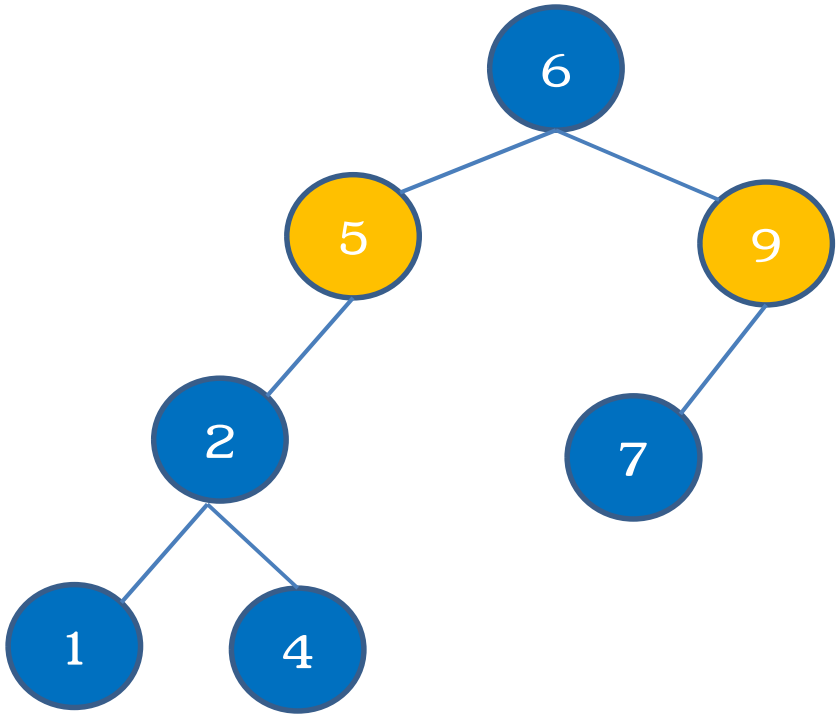
Ilustrasi hapus node leaf

- Menghapus node **1**, **4**, ataupun **7** langsung saja



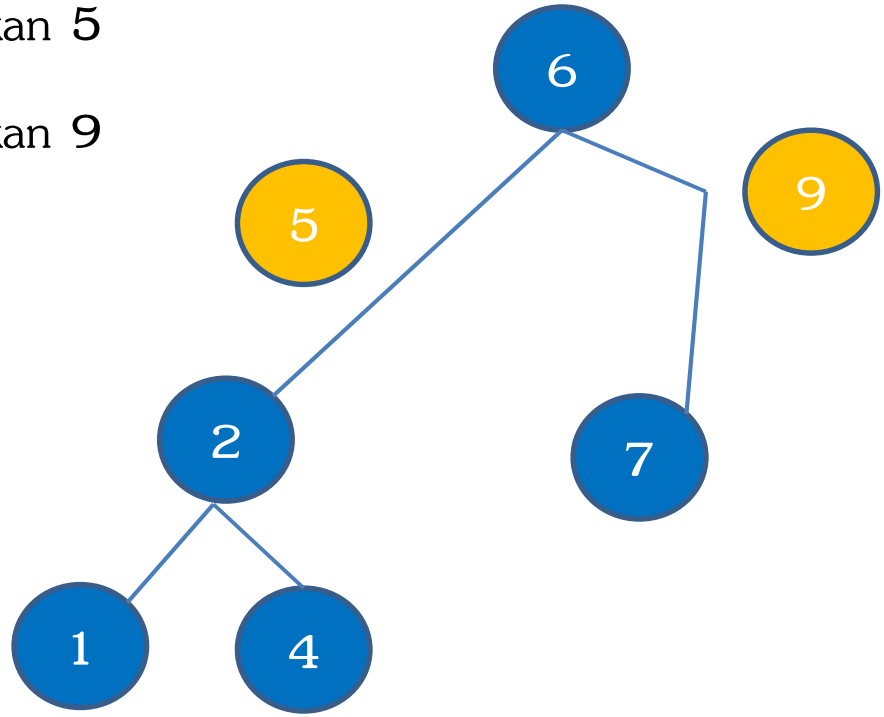
Ilustrasi Del Node dng 1 Anak

- Menghapus node bernilai 5 atau 9



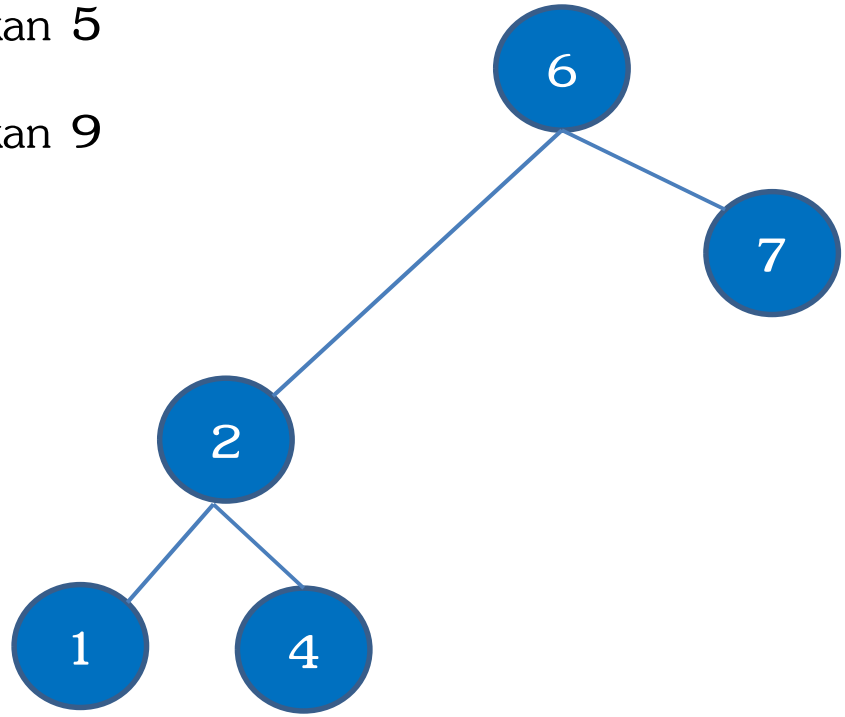
Ilustrasi Del Node dng 1 Anak

- Menghapus node bernilai 5 atau 9
  - 2 menggantikan 5
  - 7 menggantikan 9



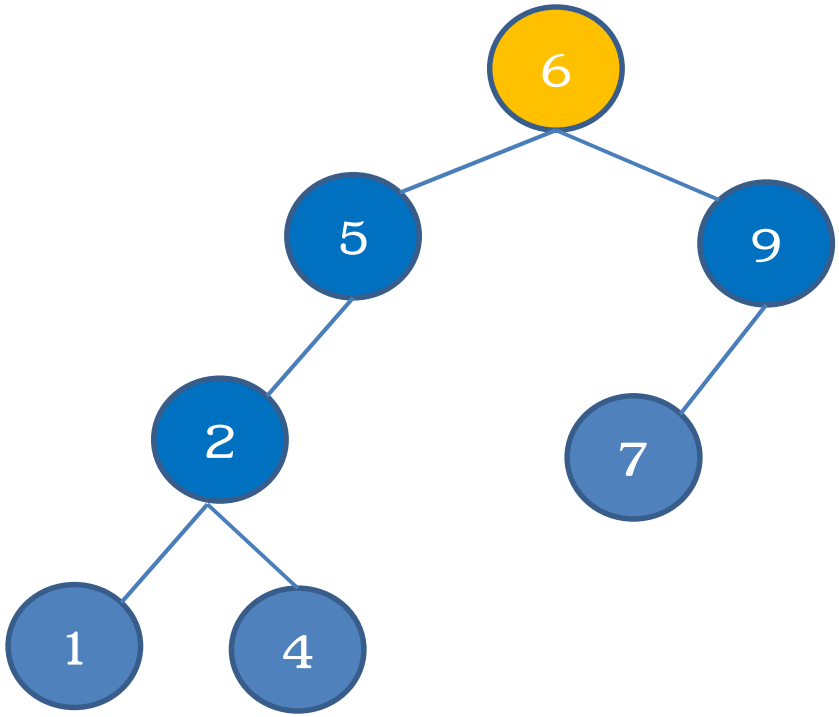
Ilustrasi Del Node dng 1 Anak

- Menghapus node bernilai 5 atau 9
  - 2 menggantikan 5
  - 7 menggantikan 9



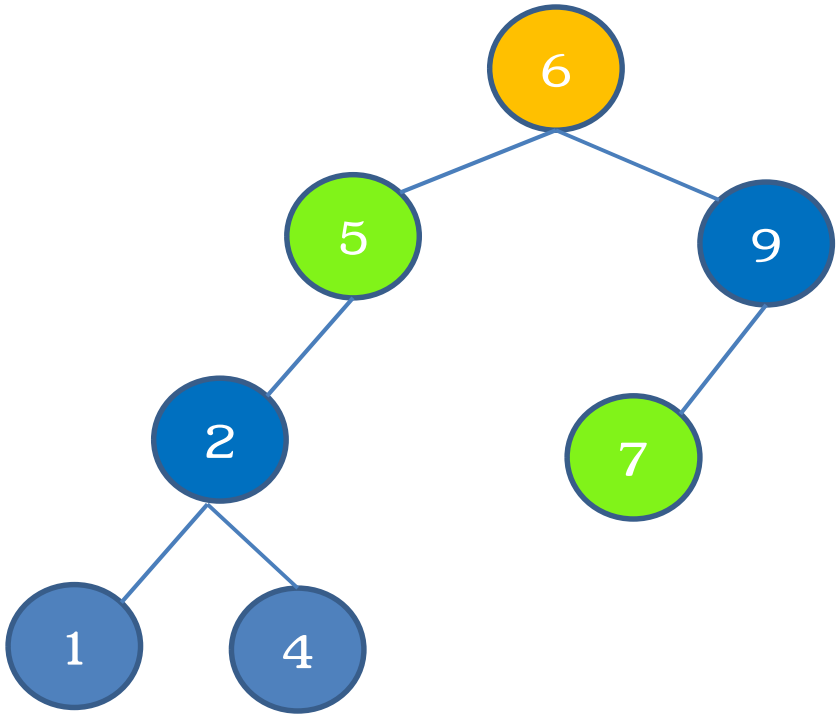
Ilustrasi Del Node dng 2 anak

- Misalkan menghapus node 6



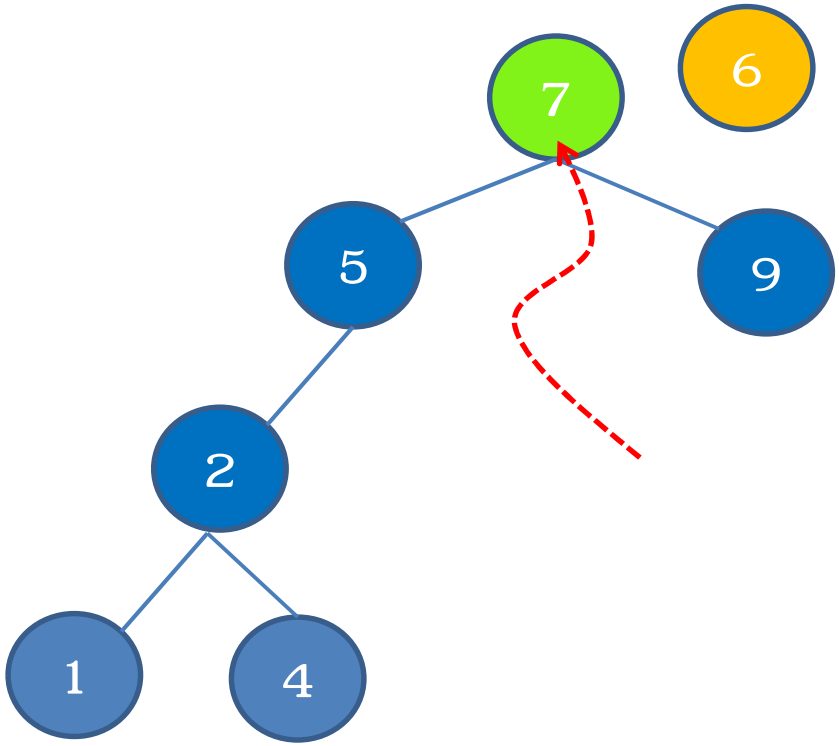
Ilustrasi Del Node dng 2 anak

- Misalkan menghapus node 6, bisa di gantikan 5 atau 7



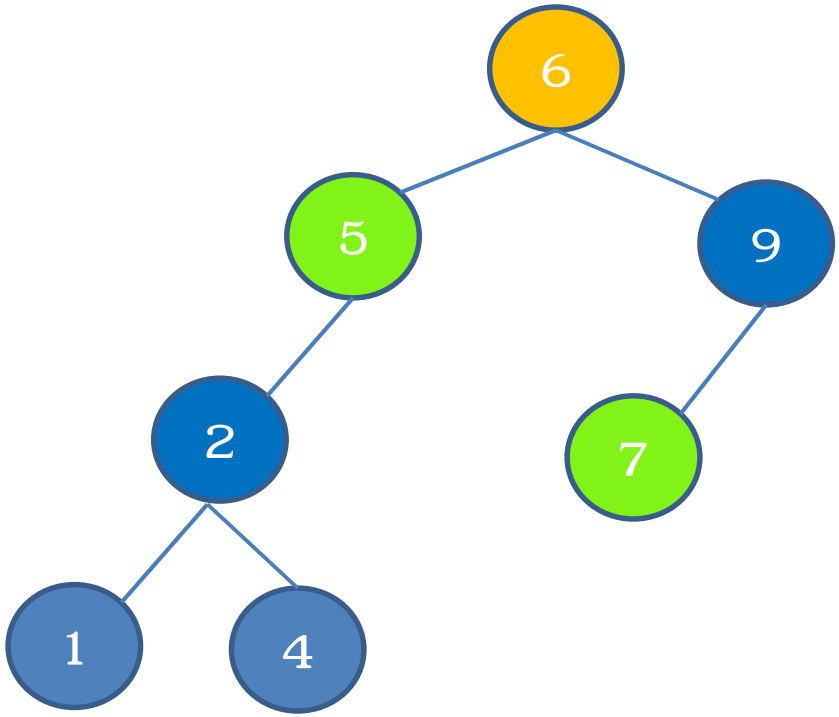
Ilustrasi Del Node dng 2 anak

- Misalkan menghapus node 6, bila di gantikan 7



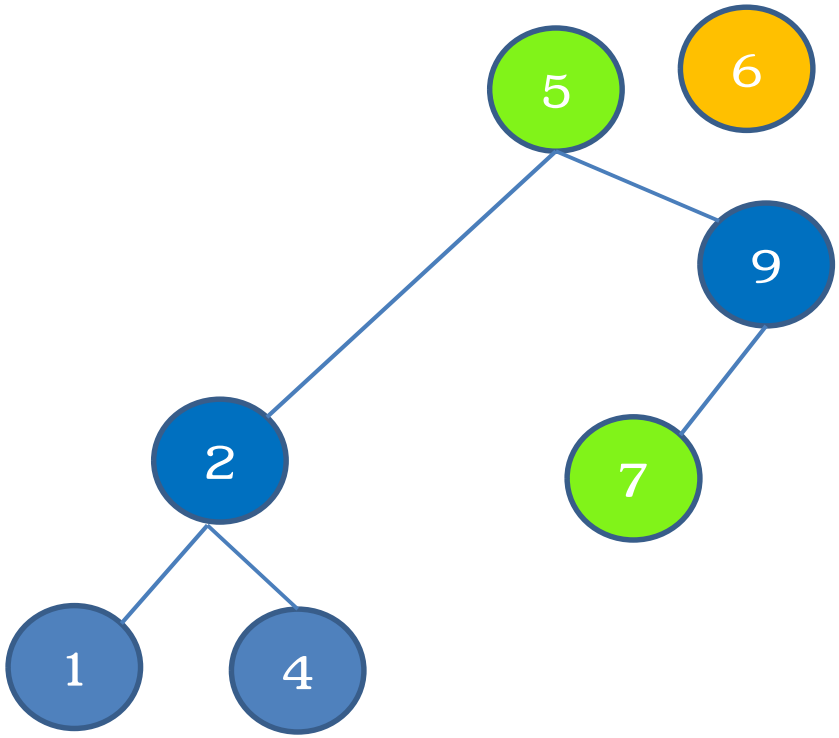
Ilustrasi Del Node dng 2 anak

- Misalkan menghapus node 6, atau bila mau digantikan 5



Ilustrasi Del Node dng 2 anak

- Misalkan menghapus node 6, atau bila mau digantikan 5

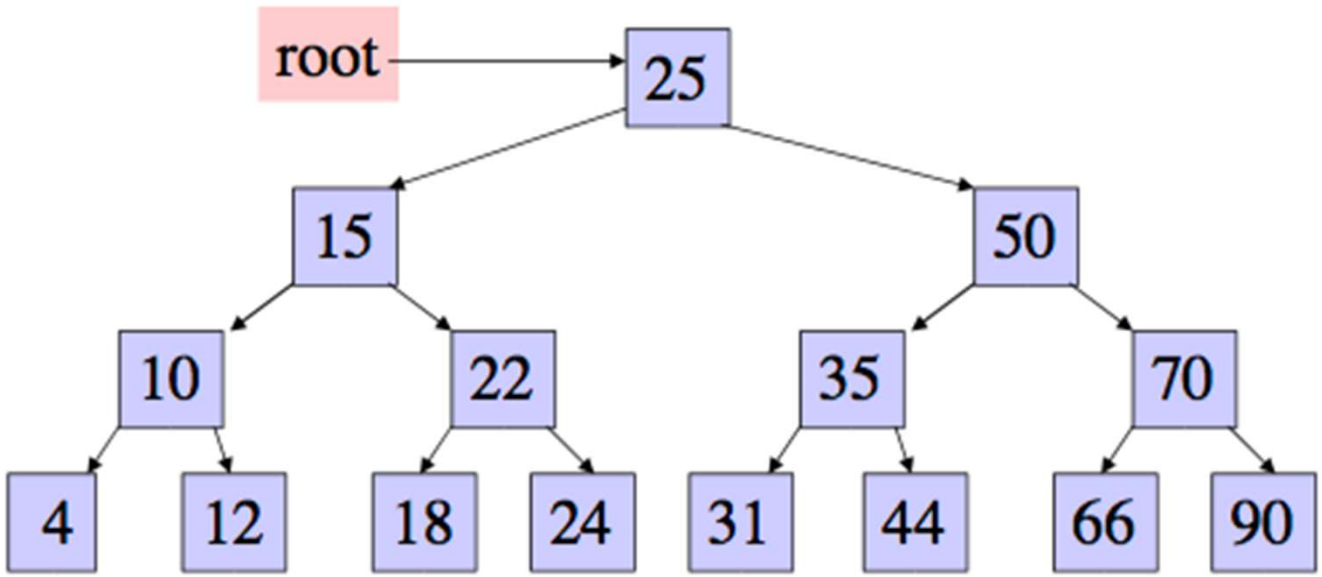


Syarat operasi

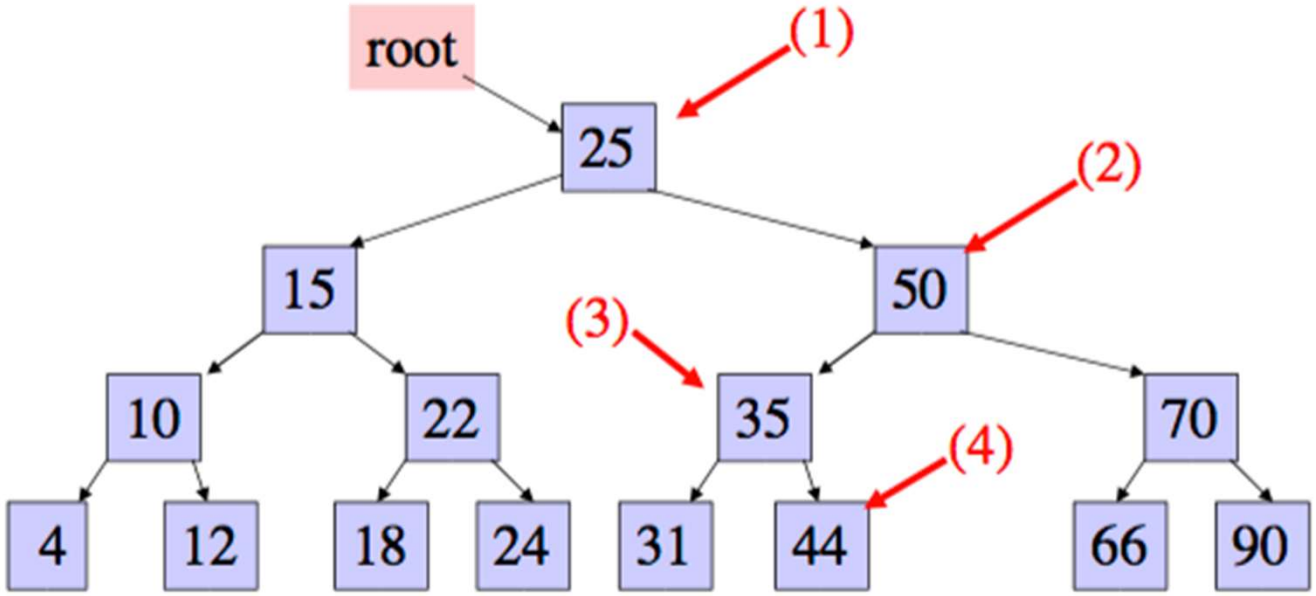
- Apapun operasinya pada BST, syarat mutlaknya adalah sifat BST harus tetap terjaga
- Jika sifat BST menjadi gugur maka operasi yang dilakukan salah
- Kompleksitas operasi
  - Penelusuran, Insert, Edit O(lg n)

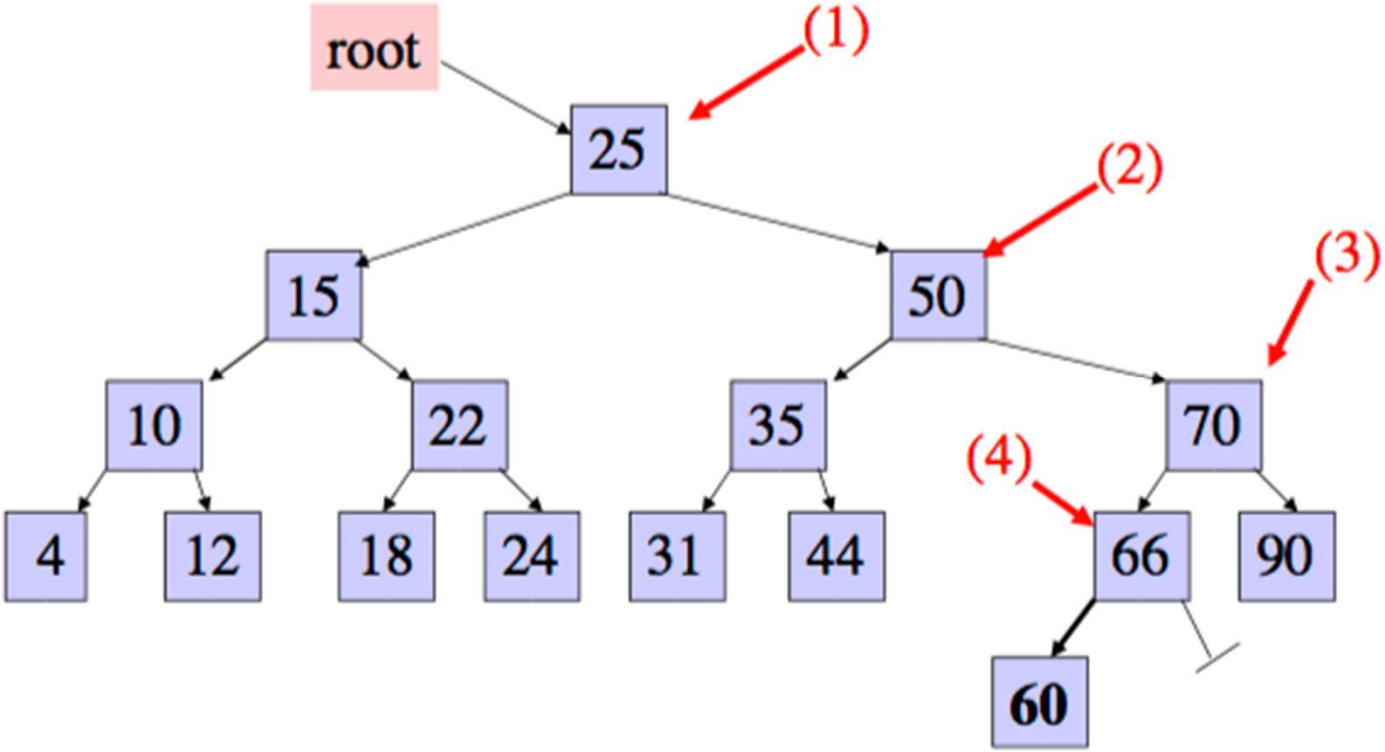
```
        (8)
       /   \
   (2)      (21)
   /   \      /
 (1)   (5 ) (13)
       /
    (3)
```

1. Hapus sebuah leaf, mis. 13
2. Hapus sebuah node dengan satu child, mis. 21
3. Hapus sebuah node dengan dua child, mis. 2



1. Cari node 45
2. Insert node 60


root
(1)
25
(2)
15
50
(3)
10
22
35
70
(4)
4
12
18
24
31
44
66
90


root
(1)
25
(2)
15
50
(3)
10
22
35
70
(4)
4
12
18
24
31
44
66
90
60

InOrder(root) visits nodes in the following order:

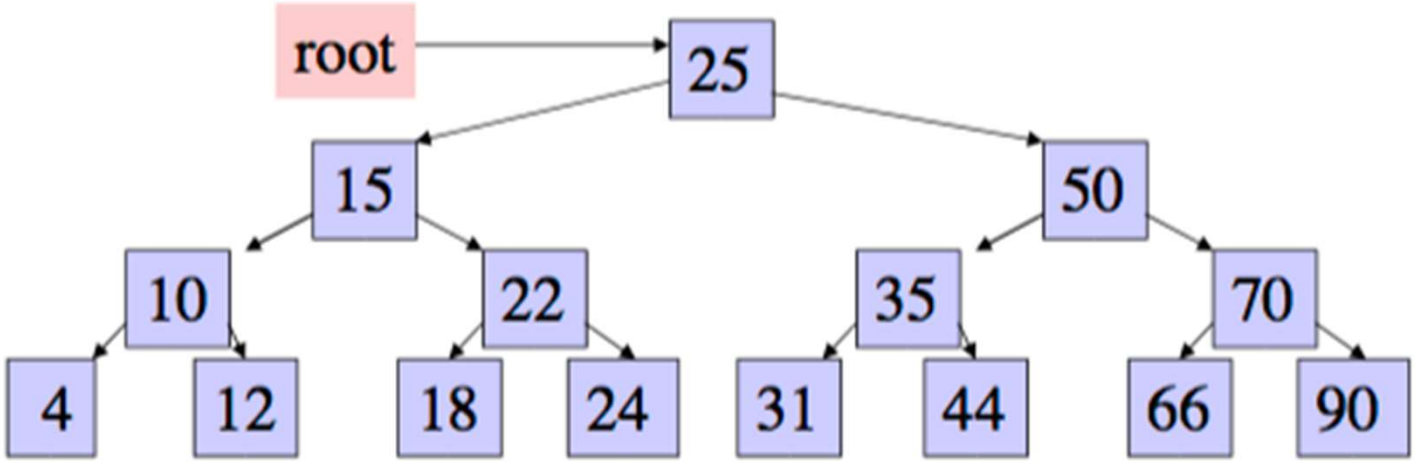
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

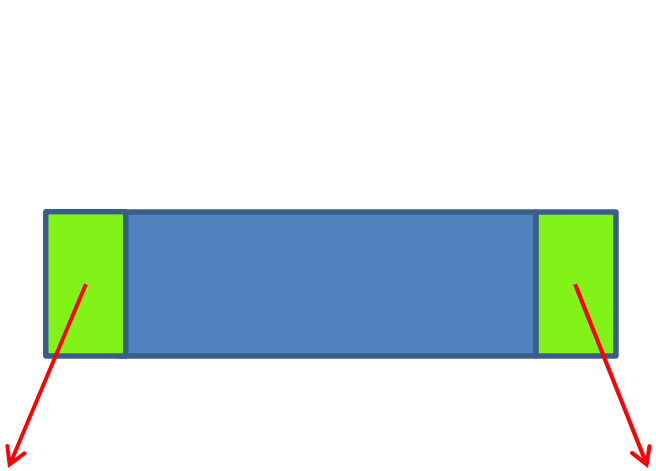
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25

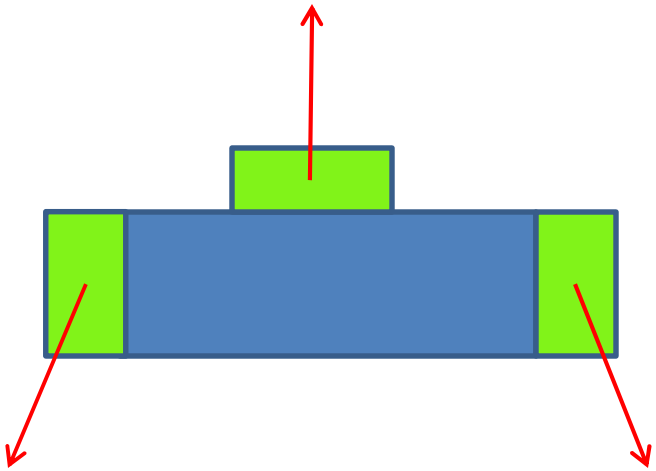


Implementasi BST

BST impl

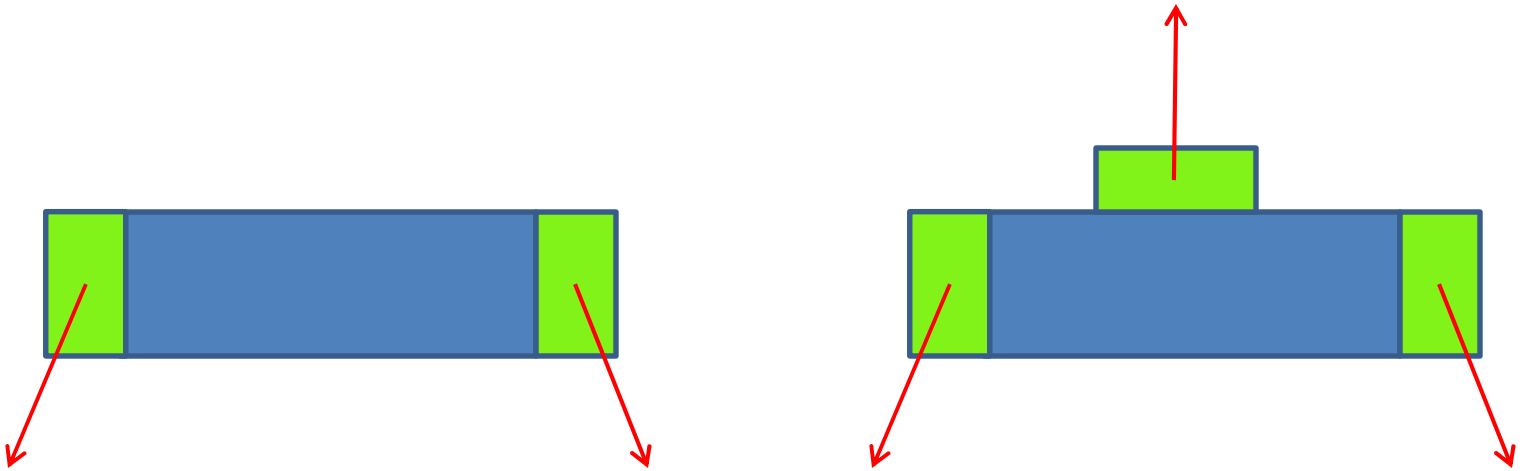
- Bentuk BST tidak selalu CBT sehingga representasinya tidak dapat digunakan array
- Salah satu solusinya gunakan node dengan pointer (bisa dua ataupun tiga)





BST impl

- Jika digunakan node dengan 2 pointer (left & right)
- Jika digunakan node dengan 3 pointer (left, right, & parent)



code

- Berikut akan dipaparkan ketika yang digunakan adalah node yang hanya dengan 2 pointer

```
typedef struct tree_node
        {
            tree_node* left;
            tree_node* right;
            int data;
        } tree_node;
```

- lengkapnya code::blocks saja yah nanti pas praktikum ☺

Pemanfaatan BST

Pemanfaatan BST

- Teknik penyimpanan indeks data bagi repository (pada teknologi awal)
  - Mungkin akan bertanya2 bahwa sebelumnya adik2 pernah mendengar teknik binary search pada array apa bedanya dengan BST ini?
    - BST ini pada kasus insert value baru O(lg n) pada insertion biasa di array cari posisi mungkin bisa O(lg n) namun setelahnya harus dilanjutkan dengan pergeseran nilai2 di kanan

LATIHAN:

1. Input : 10, 15, 25, 17, 9, 7, 8
2. Input : ORY, JFK, BRU, DUS, ZRH, MEX, ORD, NRT, ARN, GLA dan GCM

Tulis dengan tulisan tangan jawabannya dalam selembar kertas, kirim ke evieta.ipb.ac.id

Akhir slide

- Episode selanjutnya: *AVL Tree*

Sumber gambar cover

- http://homeworlddesign.com/wp-content/uploads/2014/02/Olivier-Dolle-Tree-Branch-Library-a-simple-beautiful-and-poetic-design-7.jpg